

Project Report

anonymous

1 Modeling Covid-19 Deaths and Mask Usage

1.1 Introduction

During the COVID-19 pandemic, there have been widespread discussions and debates on various preventative measures, especially on the efficacy of facial mask usage. As the number of COVID-19 deaths in Europe started to rapidly increase and new restrictions were regulated, the question of whether mask usage influences the number of COVID-19 deaths became relevant. This project investigates the relationship between average mask usage and the number of deaths (due to COVID-19) across different European countries.

There have been some studies similar to this subject (Spira B., 2022, Welsch, D., 2022), using traditional statistical methods, but not that many Bayesian approaches. The analysis in this project primarily focuses on two variables, the number of deaths per million people and the average mask usage in different European countries, at different times from 2020 to 2023. The data used is obtained from two different sources. The number of deaths is obtained from a study conducted by Mathieu et al. (2020), and the average mask usage is obtained from a study conducted by Beny Spira in 2022. The number of deaths is reported normalised as per million people since the population of different countries could otherwise impact the results greatly. The average mask usage describes the percentage of the population reporting always wearing a mask when leaving home.

To assess the relationship between COVID-19 deaths and mask usage, three models were used - a simple linear regression, a log-normal linear regression, and a hierarchical log-normal linear regression. The following parts show the program code along with explanatory text, and assess the data and models. This approach provides easy to follow structure where reader can instantly see the actual source code (most of it, at least) executed. With the models, we can try to answer the question, is there correlation between the mask usage and the deaths.

1.2 Preparation

Let us start by installing missing libraries (if any).

1.3 Load and Visualize the Data

We used Python for connecting the two different datasets we had (one contained the death tolls, dates in different countries, and one contained the average mask usage). The resulting dataset was named as “covid_data_mas.csv”. Let us load the data.

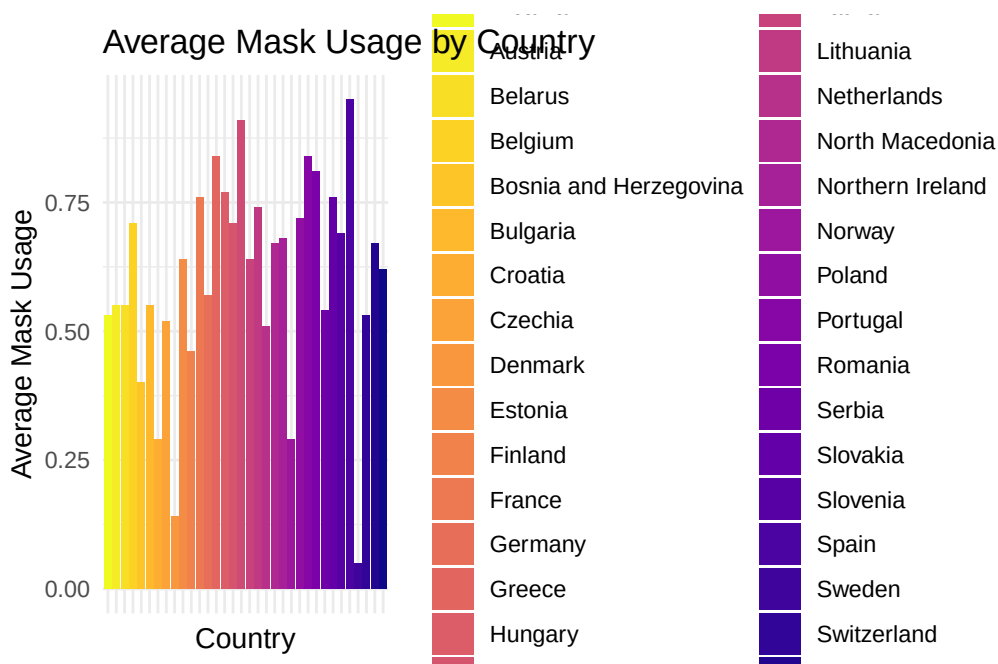
```
# Load the data
covid_data <- read.csv("data/covid_data_mask.csv")

# head(covid_data)
```

Another interesting thing to visualise is the average mask usage during the pandemic among different countries. Let us next take a look at that.

```
# Extract unique average values for each country
unique_avg_values <- covid_data %>%
  select(location, avg_mask_usage) %>%
  distinct()

# Create a plot of average mask usage by country
ggplot(unique_avg_values, aes(x = location, y =
  ↪ avg_mask_usage, fill = location)) +
  geom_bar(stat = "identity") +
  labs(title = "Average Mask Usage by Country",
       x = "Country",
       y = "Average Mask Usage") +
  scale_fill_viridis_d(option = "plasma", direction = -1) + #
  ↪ Custom viridis palette
  theme_minimal() +
  theme(axis.text.x = element_blank())
```



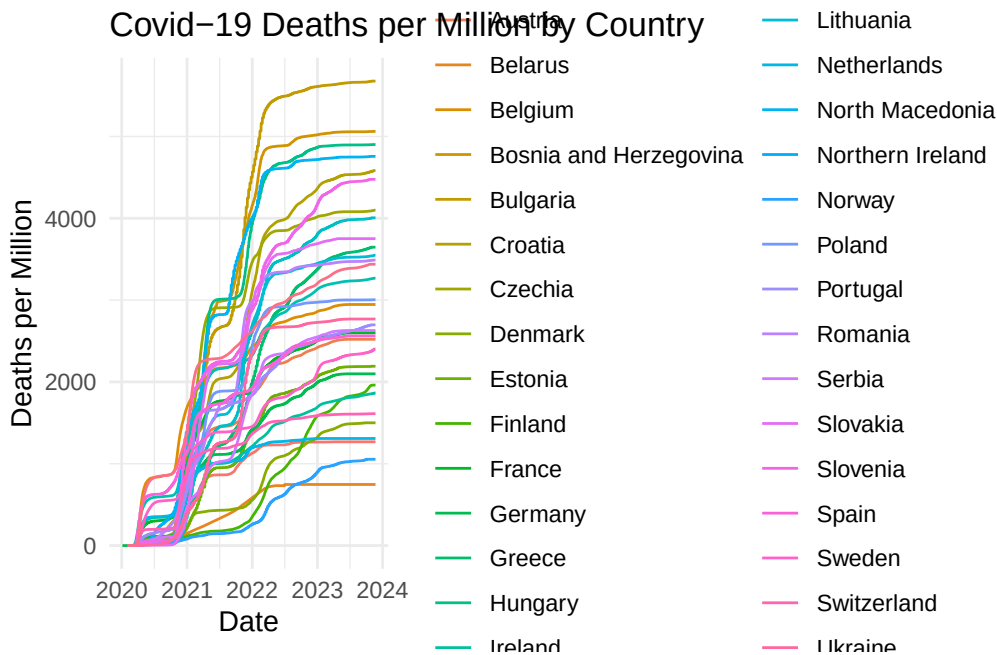
From the barplot above, we see that the mask usage really varies between countries in Europe. Looking at the plot, we can consider dividing the countries to three different classes with respect to the mask usage level.

The second variable of our interest is the number of deaths per million in different countries. Below we quickly visualize the death tolls among different countries.

```
# Convert the "date" column to a date format
covid_data$date <- as.Date(covid_data$date)

# Plot the total number of deaths per million people in
↳ different countries
ggplot(covid_data, aes(x = date, y = total_deaths_per_million,
↳ color = location)) +
  geom_line() +
  labs(title = "Covid-19 Deaths per Million by Country",
        x = "Date",
        y = "Deaths per Million") +
  theme_minimal()
```

Warning: Removed 3629 rows containing missing values (`geom\_line()`).



As we can see the amount of data is 48,204 data points which is a lot. From here on we decided to start with simpler dataset with fewer data points. This decision was also affected by the stan compiling time, since reducing the data shortens the compiling times significantly.

The data reduction and some more data cleaning (for example dropping the non needed columns and rows containing NA-values) were mostly done with python, because it is more familiar programming language for us and that's why this was much quicker task to do with it. The scripts (contains both R and Python code) for the data reduction and cleaning is included in the appendix part of this report. We split the mask usage in 3 categories w.r.t. mask usage in the countries. The category 1 included countries where mask usage was between $[0, 0.55[$, category 2 countries had mask usage between $[0.55, 0.71[$, and the category 3 had mask usages between $[0.71, 1]$. Eventually the data was reduced so that each category contains only 4 countries (randomly chosen), and the data points containing total mortality of COVID-19 deaths were listed every 20th day (instead of every day). The dates are reported as integers starting from 0.

Here we load the the data that was processed as described above (called “intersection_preprocessed_covid_data.csv”):

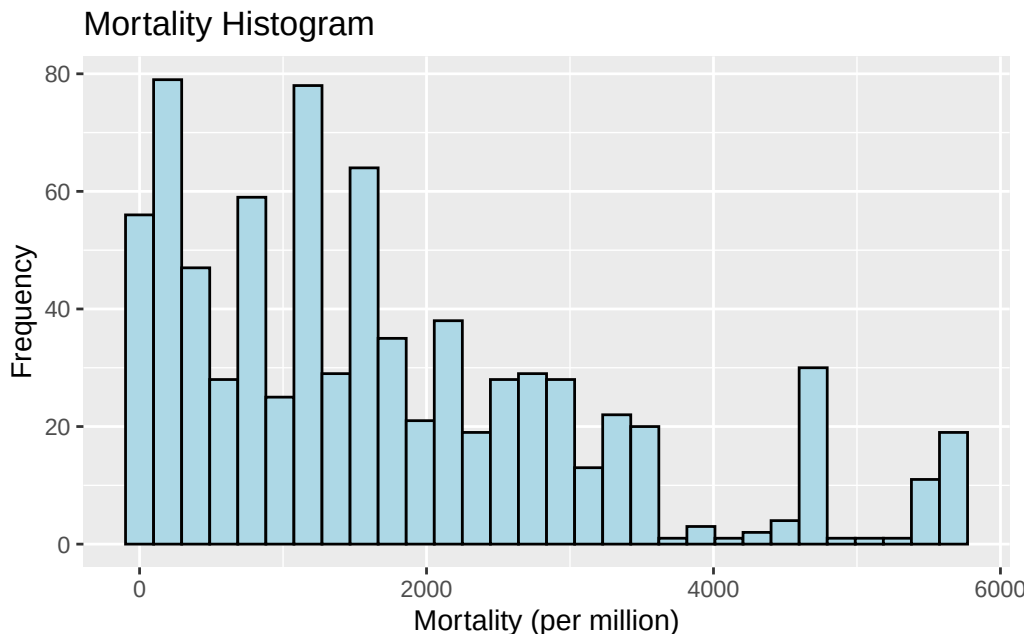
```
preprocessed_data <-  
  ↪ read.csv("data/intersection_preprocessed_covid_data.csv")  
head(preprocessed_data)
```

	X	location	date	total_deaths_per_million	mask_category
1	1082	Finland	100	43.496	1
2	1085	Lithuania	100	5.818	3
3	1087	Denmark	100	48.451	1
4	1088	North Macedonia	100	18.151	2
5	1090	Switzerland	100	130.885	1
6	1092	Belgium	100	394.907	3

Next, we plot the distribution of the deaths per million:

```
ggplot(preprocessed_data,aes(x=total_deaths_per_million)) +  
  ↪ geom_histogram(fill = "lightblue",color = "black") +  
  ↪ labs(title = "Mortality Histogram",x = "Mortality (per  
  ↪ million)",y = "Frequency")
```

`stat\_bin()` using `bins = 30`. Pick better value with `binwidth`.



Below, we take the datapoints that have date values less than *max_days*. This way we have fewer data, and it takes less time to train the models. This was done, because the amount of data still seemed to cause too long computing times.

```
max_days = 900  
data_to_use = preprocessed_data
```

```

clipped_covid_data <- data_to_use %>%
  group_by(location) %>%
  filter(date < max_days) %>%
  ungroup()

# Print or use the selected_data dataframe
print(head(clipped_covid_data))

```

```

# A tibble: 6 x 5
      X location      date total_deaths_per_million mask_category
<int> <chr>      <int>          <dbl>          <int>
1  1082 Finland      100          43.5            1
2  1085 Lithuania    100           5.82           3
3  1087 Denmark      100          48.5            1
4  1088 North Macedonia 100          18.2            2
5  1090 Switzerland   100         131.            1
6  1092 Belgium      100         395.            3

```

Let's check the number of countries in each different mask usage category:

```

# Count the number of countries in each category
count_by_category <- data_to_use %>%
  group_by(mask_category) %>%
  summarise(num_countries = n_distinct(location))

# Print the summary
print(count_by_category)

```

```

# A tibble: 3 x 2
  mask_category num_countries
      <int>      <int>
1         1         4
2         2         4
3         3         4

```

Below we transform our data to GroupedData object and convert the *mask_category* column to a factor variable for the brms fit.

```

# Transform data to GroupedData object
clipped_covid_data <- groupedData(total_deaths_per_million ~
  ↪ date | location, data = clipped_covid_data)

# as.factor(...) function is used to convert the selected
  ↪ column to a factor variable
clipped_covid_data$mask_category <-
  ↪ as.factor(clipped_covid_data$mask_category)

# Check the structure of grouped_data
head(clipped_covid_data, 10)

```

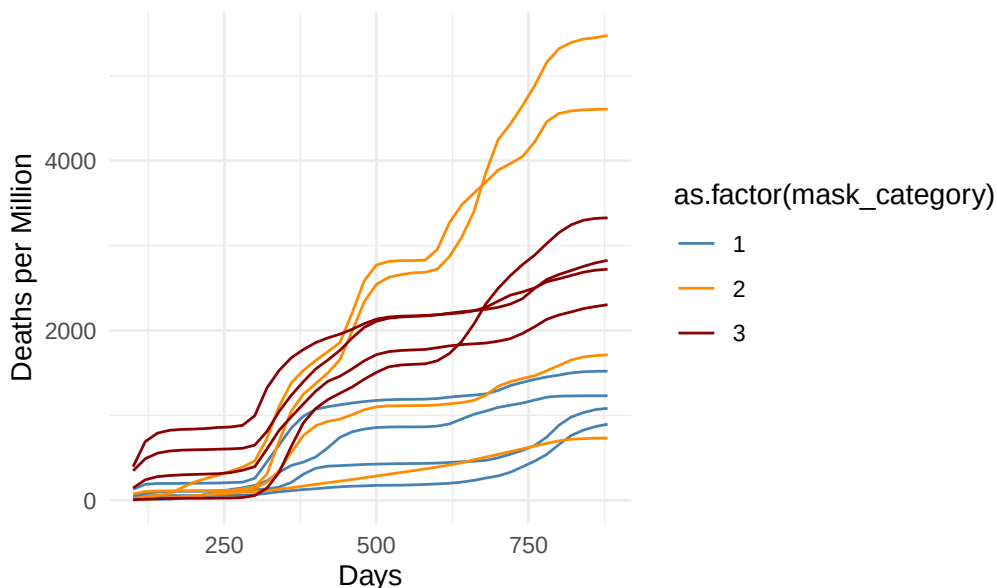
Grouped Data: total_deaths_per_million ~ date | location

	X	location	date	total_deaths_per_million	mask_category
1	1082	Finland	100	43.496	1
2	1085	Lithuania	100	5.818	3
3	1087	Denmark	100	48.451	1
4	1088	North Macedonia	100	18.151	2
5	1090	Switzerland	100	130.885	1
6	1092	Belgium	100	394.907	3
7	1094	Belarus	100	3.041	2
8	1096	France	100	144.399	3
9	1100	Bulgaria	100	4.276	2
10	1101	Italy	100	346.644	3

Before implementing any models, let us have a look at the death rates by plotting the death tolls in different countries and associate a color to each of three categories:

```
# Create a plot
ggplot(clipped_covid_data, aes(x = date, y =
  ↪ total_deaths_per_million, color =
  ↪ as.factor(mask_category), group = interaction(location,
  ↪ mask_category))) +
  geom_line() +
  labs(title = "Number of Deaths per Million by Country and
  ↪ Mask Category",
  x = "Days",
  y = "Deaths per Million") +
  scale_color_manual(values = c("1" = "steelblue", "2" =
  ↪ "darkorange", "3" = "darkred")) + # Adjust colors as
  ↪ needed
  theme_minimal()
```

Number of Deaths per Million by Country and Mask Category



1.4 Models

1.4.1 Simple linear regression

Let us now fit our first model, which is the simple linear regression. Below on the first few lines of the code you can see the priors used for the model predictor parameters. These priors we're decided by simply by looking at the visualisations above, and by intuition. For the prior on date, we tried to deduce the amount of increase in deaths per day, and for the effects of mask usages, we tried to deduce how much average mortality difference would be possible between different mask usage categories.

```
custom_priors <- c(
  prior(normal(0, 5), coef = "date"),
  prior(normal(0, 200), coef = "mask_category2"),
  prior(normal(0, 200), coef = "mask_category3")
)

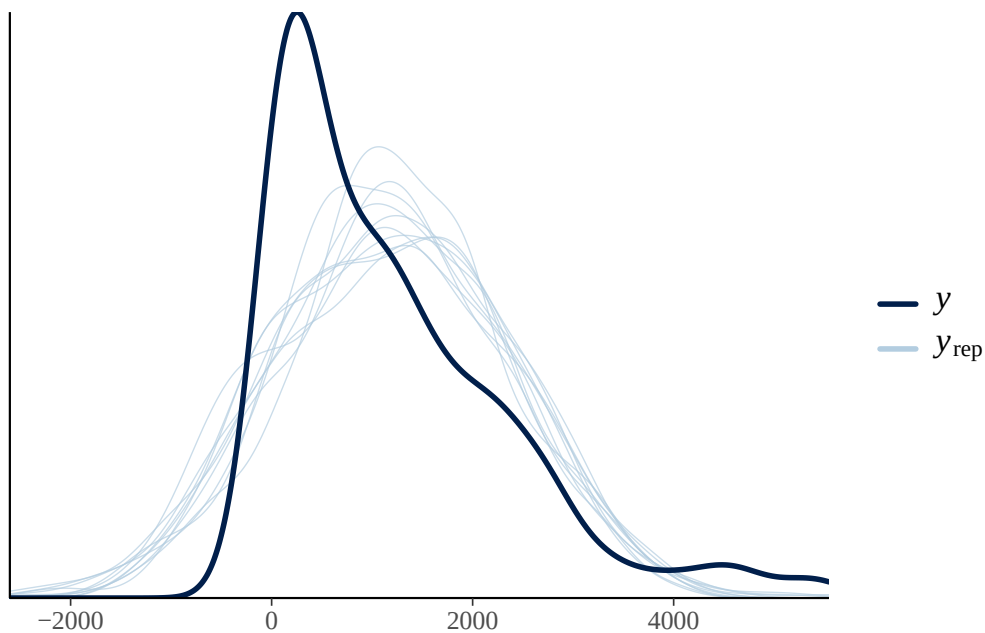
suppressMessages(
  custom_fit <- brms::brm(
    # This specifies the formula
    total_deaths_per_million ~ 1 + mask_category + date,
    # This specifies the dataset
    data = clipped_covid_data,
    # This specifies the observation model family
    family = "gaussian",
    # This passes the priors specified above to brms
    prior = custom_priors,
    # This causes brms to cache the results
    file = "additional_files/covid_data_custom_fit",

    # Use these for parallel processing
    cores = 8,
    threads = threading(8)
  )
)
```

After fitting the model, we can use the *pp_check* function to check the posterior predictions in relation to the observed data. In the plot below the *y* values are the observed data, and the *y_{rep}* values are replicated data sets from the posterior predictive distribution. Let's next run the posterior predictive check for the fitted linear regression model.

```
pp_check(custom_fit)
```

Using 10 posterior draws for ppc type 'dens_overlay' by default.



From the plot above, we see that the model fits the observed data pretty badly. There is lot to improve!

Let us also run another kind of posterior predictive check and plot the results. In the following, each mask category group is plotted separately.

```
pp_plot <- pp_check(custom_fit, type = "intervals_grouped",
  ↪ group = "mask_category") +
  theme_light() # Use theme_light to make the background
  ↪ lighter
```

Using all posterior draws for ppc type 'intervals_grouped' by default.

```
# Save the plot to a file with specified dimensions
ggsave("pp_check_plot.png", plot = pp_plot, height = 10, width
  ↪ = 25)
```

Here are the plots obtained from the second `pp_check`:

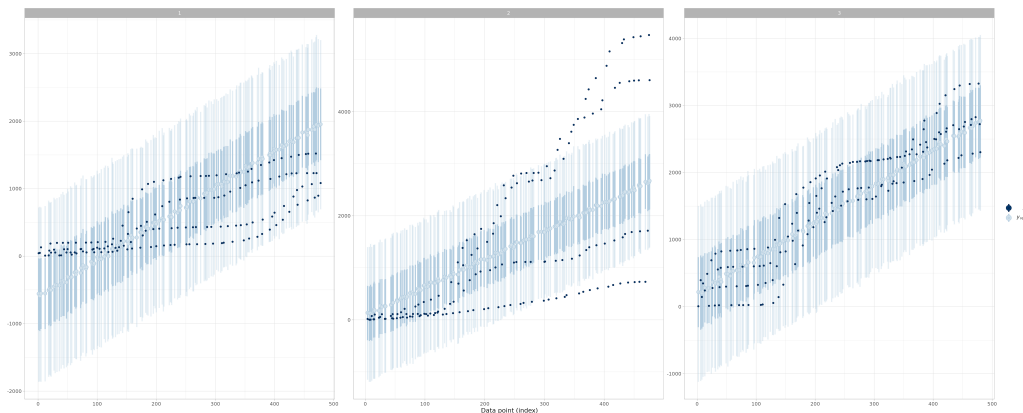


Figure 1: Model 1 (Linear regression)

As we can see, the linear model is unable to capture the non-linear nature of the actual observed data. The 3. category seems to be captured best by our model, but overall all the categories are badly captured.

Let us finally check the summary of the model with the function *summary*. (NB! The results are assessed later below!).

```
summary(custom_fit)
```

```
Family: gaussian
Links: mu = identity; sigma = identity
Formula: total_deaths_per_million ~ 1 + mask_category + date
Data: clipped_covid_data (Number of observations: 480)
Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
       total post-warmup draws = 4000
```

Population-Level Effects:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
Intercept	-880.34	96.99	-1072.92	-692.96	1.00	4124	3135
mask_category2	682.62	80.27	520.42	838.53	1.00	4136	3229
mask_category3	781.89	81.14	621.57	939.77	1.00	3773	3114
date	3.23	0.15	2.92	3.52	1.00	5167	2876

Family Specific Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sigma	783.31	25.87	734.67	835.44	1.00	4342	3041

Draws were sampled using `sample(hmc)`. For each parameter, Bulk_ESS and Tail_ESS are effective sample size measures, and Rhat is the potential scale reduction factor on split chains (at convergence, Rhat = 1).

Below, using the function `plot`, we can assess how our model converged and if it fits the data well (NB! The results are assessed later below!).

```
png(file="convergence_1_1.png",
width=1100, height=1100)
plot(custom_fit)[1]
```

```
[[1]]
```

1.4.2 Hierarchical linear regression

The model above used gaussian observation family, which is obviously over-simplified assumption to use with our covid data. We could try to use lognormal observation model, which only allows positive values. This makes sense since the number of deaths can not be negative. We also have another option to try out. We can make hierarchical model by including both population and group level effects in to the model. This way the model is improved by directly considering potential differences in death rate for individual countries. Some countries may have more drastic/faster death rates than others due to reasons that are not included in our data. Since it is advised to change only one or a few things about

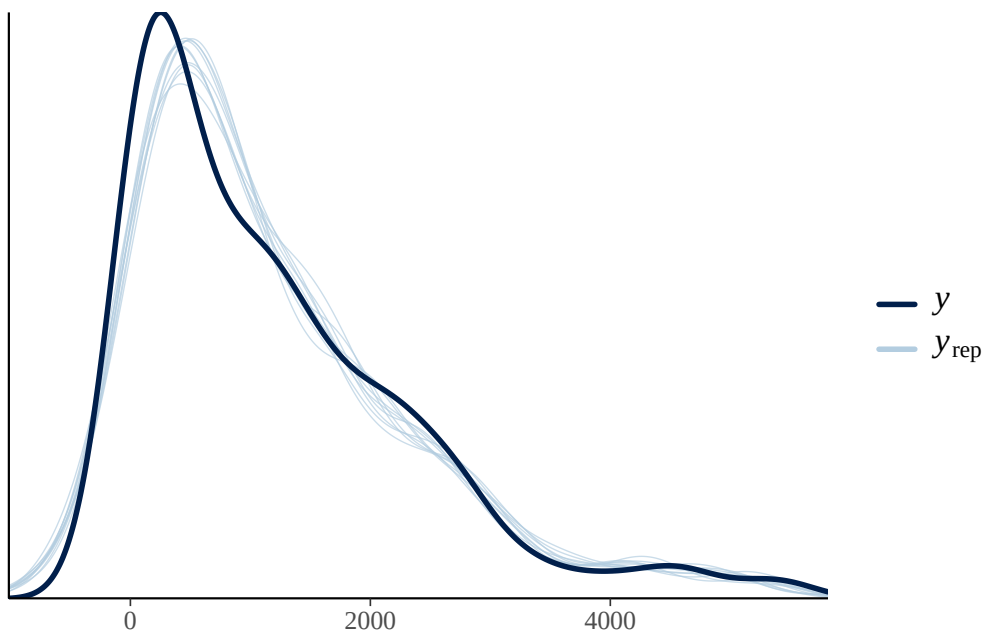
a model at once, let us first try to do this hierarchical model before modifying the observation family.

```
suppressMessages(  
  custom_fit2 <- brms::brm(  
    # This specifies the formula  
    total_deaths_per_million ~ 1 + mask_category + date +  
    ↪ (date|location), # include a group effect in brms  
    # This specifies the dataset  
    data = clipped_covid_data,  
    # This specifies the observation model family  
    family = "gaussian", # Could be improved by trying  
    ↪ "lognormal",  
    # This passes the priors specified above to brms  
    prior = custom_priors,  
    # This causes brms to cache the results  
    file = "additional_files/covid_data_custom_fit2",  
  
    # Use these for parallel processing  
    cores = 8,  
    threads = threading(8)  
  )  
)
```

Again, we run the posterior predictive check for the fitted hierarchical model.

```
brms::pp_check(custom_fit2)
```

Using 10 posterior draws for ppc type 'dens_overlay' by default.



From the plot above, we see that the model fits the observed data a lot better than the first model, though this improvement didn't fix the problem of including

negative values in the model. Overall, the results seem promising, but some improvements can be probably made (such as changing the observation family to exclude negative values).

Again, we also run the second `pp_check` where each mask category group is plotted separately.

```
pp_plot <- pp_check(custom_fit2, type = "intervals_grouped",  
  ↪ group = "mask_category") +  
  theme_light() # Use theme_light to make the background  
  ↪ lighter
```

Using all posterior draws for ppc type 'intervals_grouped' by default.

```
# Save the plot to a file with specified dimensions  
ggsave("customfit2_pp_check_plot.png", plot = pp_plot, height  
  ↪ = 10, width = 25)
```

Here are the plots obtained from the second `pp_check`:

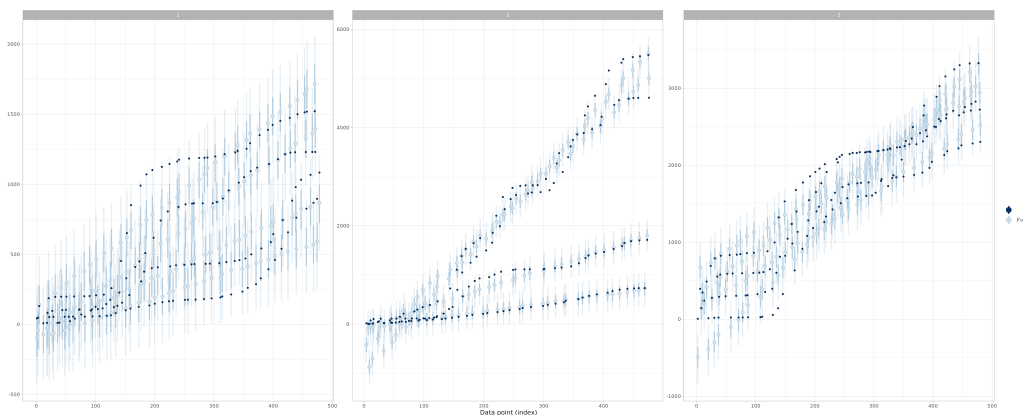


Figure 2: Model 2 (Hierarchical log-normal linear regression)

Here we can see, that the the second model is fitting the observe data better, but again is not able to capture well the non-linearity of the data. The 2. category seems to be captured best by our model, but overall all the categories aren't that well captured.

Finally, let's check the summary of the model (NB! The results are assessed later below!).

```
summary(custom_fit2)
```

Warning: There were 16 divergent transitions after warmup. Increasing `adapt_delta` above 0.8 may help. See <http://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup>

```
Family: gaussian  
Links: mu = identity; sigma = identity  
Formula: total_deaths_per_million ~ 1 + mask_category + date + (date | location)
```

```
Data: clipped_covid_data (Number of observations: 480)
Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
      total post-warmup draws = 4000
```

Group-Level Effects:

~location (Number of levels: 12)

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sd(Intercept)	598.98	143.80	382.01	953.91	1.00	1333	2040
sd(date)	2.51	0.55	1.67	3.78	1.00	1307	2075
cor(Intercept,date)	-0.80	0.13	-0.95	-0.45	1.00	2302	2189

Population-Level Effects:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
Intercept	-412.89	189.61	-787.74	-35.95	1.00	1505	2178
mask_category2	-67.26	155.61	-373.91	236.73	1.00	2673	2537
mask_category3	185.99	150.90	-121.51	473.93	1.00	2204	2194
date	3.12	0.71	1.64	4.46	1.00	1351	1976

Family Specific Parameters:

	Estimate	Est.Error	l-95% CI	u-95% CI	Rhat	Bulk_ESS	Tail_ESS
sigma	195.96	6.47	183.69	208.86	1.00	4838	2857

Draws were sampled using `sample(hmc)`. For each parameter, `Bulk_ESS` and `Tail_ESS` are effective sample size measures, and `Rhat` is the potential scale reduction factor on split chains (at convergence, `Rhat` = 1).

Below, using the function `plot`, we can assess how our model converged and if it fits the data well (NB! The results are assessed later below!).

```
png(file="convergence_2_1.png",
width=1100, height=1100)
plot(custom_fit2)[1]
```

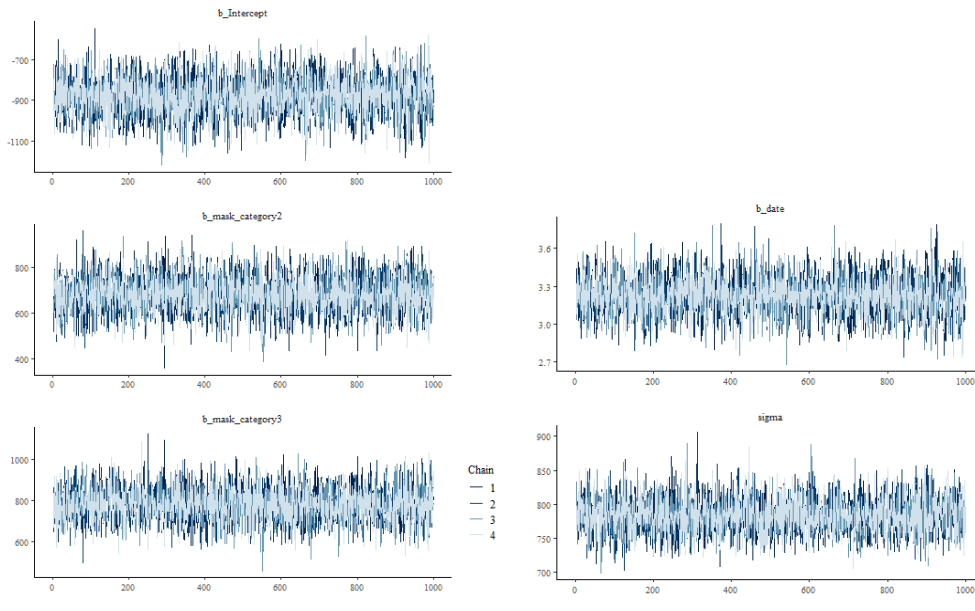
```
[[1]]
```

1.4.3 Convergence, rhat, and effective sample size diagnostics and interpretation

We can estimate the convergences of the models by looking at the diagnostics outputted by the summaries of the models. One metric of convergence is the $\hat{R}$. All $\hat{R}$ values are 1.0 for both of the models, which indicates us that the models have converged. That is why we do not see reason to run more iterations.

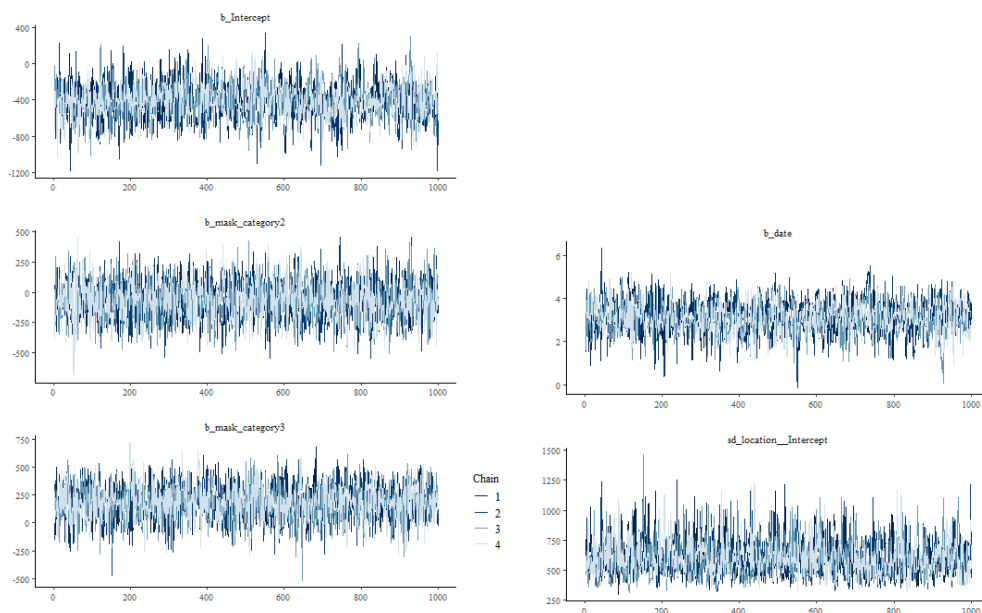
If we look at our summaries, we get a bit of information about effective sample sizes. The `Bulk_ESS` and `Tail_ESS` are the effective sample size measures for each parameter. These should be high (>1000) to be correct. All of our models have all ESS values of over 1000, meaning that the accuracy of importance sampling is good with both models.

Let us next take a look at the chains of the first model plotted with `plot` function.



Through visual convergence evaluation, it seems that the chains have converged pretty well.

Let us then take a look at the chains of the second model.



Through visual convergence evaluation, the chains seem similar to the ones from the first model, thus we conclude that the chains have converged well.

1.4.4 Model comparison using the ELPD

Let us compare the two models w.r.t the log-score. Below we first run leave-one-out (LOO) cross-validation for our two models, and print out the resulting scores. Finally, we compare the results of the two cross-validations.

```
loo1 <- loo(custom_fit)
loo2 <- loo(custom_fit2)

print(loo1)
```

Computed from 4000 by 480 log-likelihood matrix

```
      Estimate   SE
elpd_loo -3881.9 23.4
p_loo      6.2  0.9
looic     7763.7 46.8
-----
```

Monte Carlo SE of elpd_loo is 0.0.

All Pareto k estimates are good ($k < 0.5$).
See `help('pareto-k-diagnostic')` for details.

```
print(loo2)
```

Computed from 4000 by 480 log-likelihood matrix

```
      Estimate   SE
elpd_loo -3227.6 20.9
p_loo      25.8  3.3
looic     6455.2 41.7
-----
```

Monte Carlo SE of elpd_loo is 0.1.

Pareto k diagnostic values:

		Count	Pct.	Min. n_eff
$(-\infty, 0.5]$	(good)	478	99.6%	1247
$(0.5, 0.7]$	(ok)	2	0.4%	138
$(0.7, 1]$	(bad)	0	0.0%	<NA>
$(1, \infty)$	(very bad)	0	0.0%	<NA>

All Pareto k estimates are ok ($k < 0.7$).
See `help('pareto-k-diagnostic')` for details.

Compare the results:

```
comparison <- loo_compare(loo1, loo2)

print(comparison)
```

```
      elpd_diff se_diff
custom_fit2    0.0     0.0
custom_fit  -654.3    29.8
```

We see that the second model (`custom_fit2`) has the highest expected log predictive density (probability) indicating the best fit among the two models.

1.4.5 Prior sensitivity

It is useful to check, how sensitive the models are for changes in prior distribution. In this section we try to fit our two types of models, but this time we do few tries with different priors. Let us try out two different standard deviation values for all of the three priors, one that gives us a “wider prior”, and one that gives us a “narrower” prior distribution. We exhaustively try out all the possible combinations of the priors and check the results.

Let us first do this kind of exhaustive check of different priors for the first model (i.e the simple linear regression model):

```
# set run TRUE if needed

run = FALSE

if (run == TRUE){
  # Define a range of priors for 'date' coefficient
  prior_ranges_date <- list(
    prior(normal(0, 10), coef = "date"),      # Wider prior
    prior(normal(0, 2), coef = "date")        # Narrower prior
  )

  # Define a range of priors for 'mask_category2' coefficient
  prior_ranges_mask2 <- list(
    prior(normal(0, 400), coef = "mask_category2"),      #
    ↪ Wider prior
    prior(normal(0, 100), coef = "mask_category2")      #
    ↪ Narrower prior
  )

  # Define a range of priors for 'mask_category3' coefficient
  prior_ranges_mask3 <- list(
    prior(normal(0, 400), coef = "mask_category3"),      #
    ↪ Wider prior
    prior(normal(0, 100), coef = "mask_category3")      #
    ↪ Narrower prior
  )

  # Fit models with different priors
  models <- list()
  i <- 1
  for (prior_date in prior_ranges_date) {
    for (prior_mask2 in prior_ranges_mask2) {
      for (prior_mask3 in prior_ranges_mask3) {
        priors <- c(prior_date, prior_mask2, prior_mask3)
        suppressMessages(
          model <- brm(
            total_deaths_per_million ~ 1 + mask_category +
            ↪ date,
            data = clipped_covid_data,
            family = "gaussian",
            prior = priors,

```

```

      # For parallel processing
      cores = 8,
      threads = threading(8)
    )
  )

  models[[i]] <- pp_check(model)
  i <- i + 1
}
}
}

gridExtra::grid.arrange(grobs = models)
}

```

Let's then do some visualising of the posterior predictive checks across models with different priors:

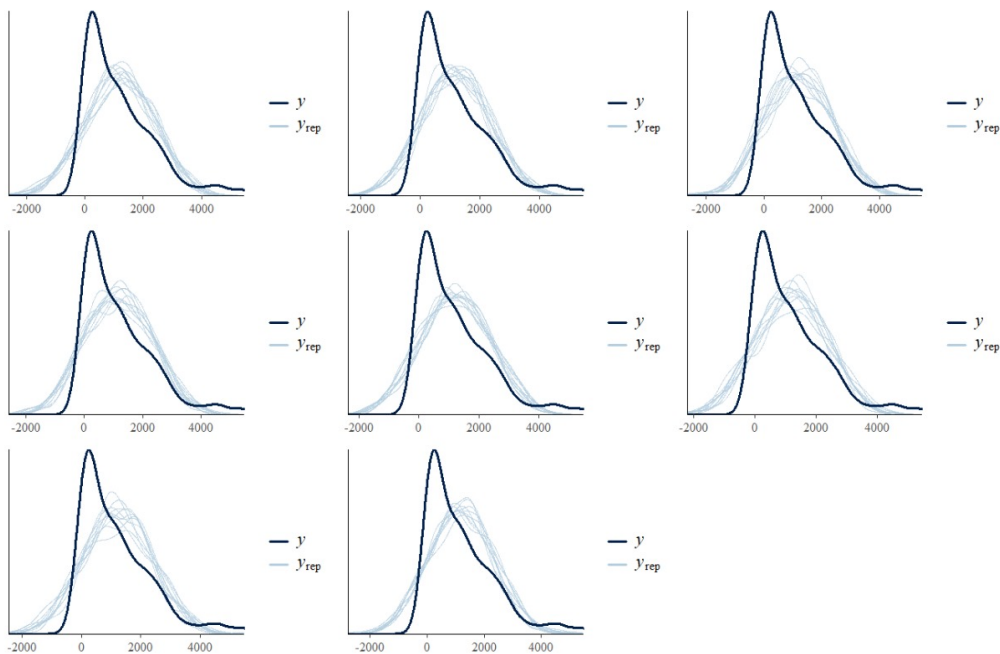


Figure 3: Model 1 linear Regression prior sensitivity analysis

We see that all of the different combinations lead to almost identical result as with the first “original” model. We thus conclude that the first model is very robust for changes in prior distribution.

We then repeat the same process for the second model (i.e. the hierarchical linear regression model):

```

# Fit models with different priors for our hierarchical model

if (run == TRUE){

  models_2 <- list()

```



```

i <- 1
for (prior_date in prior_ranges_date) {
  for (prior_mask2 in prior_ranges_mask2) {
    for (prior_mask3 in prior_ranges_mask3) {
      priors <- c(prior_date, prior_mask2, prior_mask3)
      suppressMessages(
        model <- brm(
          total_deaths_per_million ~ 1 + mask_category + date
          ↪ + (date|location), # include a group effect in brms
          data = clipped_covid_data,
          family = "gaussian",
          prior = priors,

          # For parallel processing
          cores = 8,
          threads = threading(8)
        )
      )

      models_2[[i]] <- pp_check(model)
      i <- i + 1
    }
  }
}

gridExtra::grid.arrange(grobs = models_2)
}

```

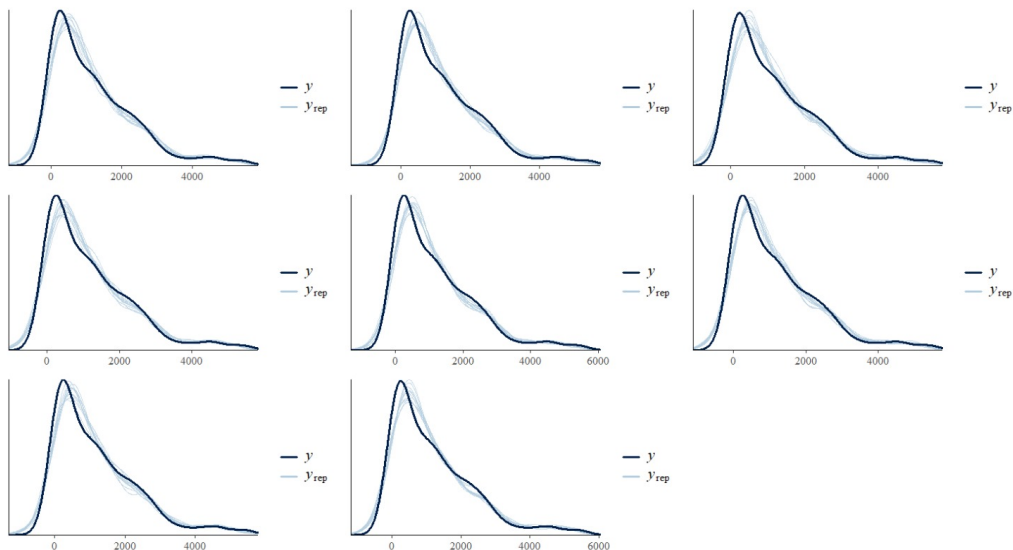


Figure 4: Model 2 hierarichal linear Regression prior sensitivity analysis

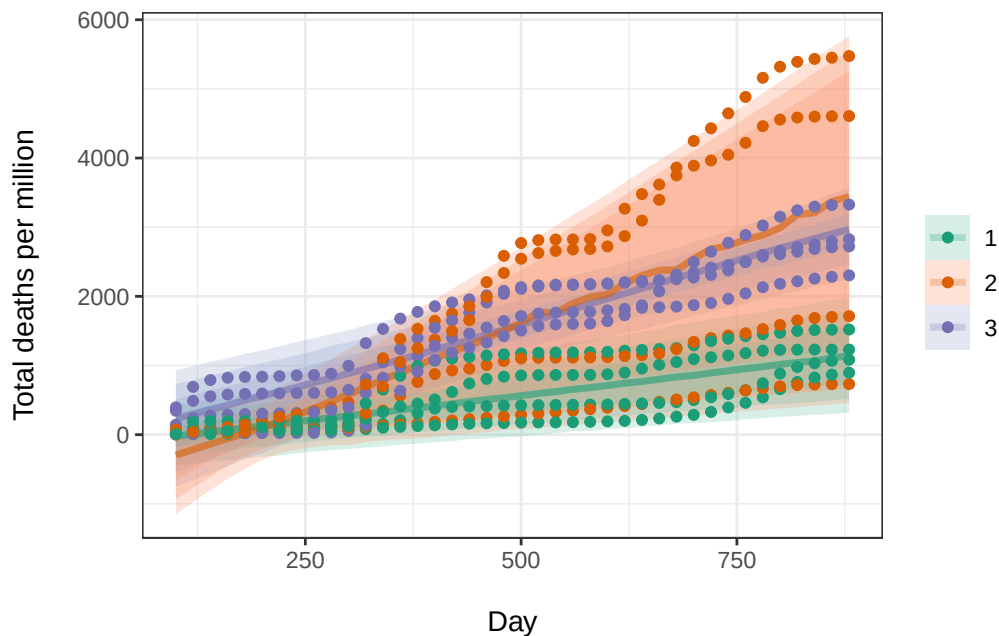
Exactly the same thing can be said, that was said with the first model. We see that all of the different combinations lead to almost identical result as with the first “original” model. The second model also seems very robust for changes in prior distribution.

1.5 Discussion

The linear models with gaussian observation family did not capture the observed data as well as we wanted. We could have tried lognormal model, since the deaths rates per counties does not seem to be linear, and the death rates cannot be negative and the lognormal does not allow negative values. We could also try different families and priors, for example better informed priors could be obtained from literature.

With the following code we can try to compare the difference between the three different categories of mask usage using our second model (which had better fit). We scatter plot the actual observed data, and the model's predictions we plot with a linear ribbon around a regression line, and the width of the ribbon varies as the function of .95, .80, .5 quantiles.

```
(category_fit <- clipped_covid_data %>%
  group_by(mask_category) %>%
  add_predicted_draws(custom_fit2) %>%
  ggplot(aes(x = date, y = total_deaths_per_million, color =
    ↪ ordered(mask_category), fill = ordered(mask_category)))
    ↪ +
  stat_linerribbon(aes(y = .prediction), .width = c(.95, .80,
    ↪ .50), alpha = 1/4) +
  geom_point(data = clipped_covid_data) +
  scale_fill_brewer(palette = "Set2") +
  scale_color_brewer(palette = "Dark2") +
  theme_bw() +
  ylab("Total deaths per million\n") +
  xlab("\nDay") +
  theme_bw() +
  theme(legend.title = element_blank()))
```



1.6 Conclusion

Looking at the previous plot, interestingly, it seems that the death rate is the lowest among the countries with the least mask usage, and highest among the second group. Obviously, we don't have enough data to make any reliable conclusions about this data, but the results here give reason to at least further investigate the causality between mask usage and the death rates. In the matter of fact, causality may have big role in the results we see above. Indeed, it might be possible that a country that has already seen high death rates is more willingly to obey the mask recommendations set by the government. We have to also keep in mind that the mask usage is not the only indicator, since politics and population and countries wealth should also take account when making this kind of modelling.

Working on this project, we learned several things. We realised that half of the analysis process is dedicated to getting the data into a proper form for analysis. In our case, the prior choices did not matter as much as expected, which was surprising. Additionally, we gained insights into the challenges and errors encountered when adjusting the model, along with a general workflow for Bayesian data analysis. Considering the reporting accuracy, we only used integers in our model (priors, etc) and we cannot change the outputs of *summary* or other similar functions. But when considering and comparing the numbers, we used roughly 1 decimal, since our models weren't very accurate.

1.7 References

(AI usage: ChatGPT 3.5 used for helping with code syntax (R syntax is not so familiar to us, so we needed a helping hand) and checking grammar.)

Edouard Mathieu, Hannah Ritchie, Lucas Rod  s-Guirao, Cameron Appel, Charlie Giattino, Joe Hasell, Bobbie Macdonald, Saloni Dattani, Diana Beltekian, Esteban Ortiz-Ospina and Max Roser (2020) - "Coronavirus Pandemic (COVID-19)". Published online at OurWorldInData.org. Retrieved from: 'https://ourworldindata.org/coronavirus' [Online Resource]

Spira B. (2022). Correlation Between Mask Compliance and COVID-19 Outcomes in Europe. *Cureus*, 14(4), e24268. <https://doi.org/10.7759/cureus.24268>

Welsch, D. (2022) The Impact of Mask Usage on COVID-19 Deaths: Evidence from US Counties Using a Quasi-Experimental Approach. *The B.E. Journal of Economic Analysis & Policy*, Vol. 22 (Issue 1), pp. 1-28. <https://doi.org/10.1515/bejeap-2021-0157>

https://mc-stan.org/rstanarm/reference/pp_check.stanreg.html

<https://ourcodingclub.github.io/tutorials/brms/#part2>

https://mc-stan.org/docs/2_18/reference-manual/effective-sample-size-section.html

Course material

Stan manual

R manual

1.8 Appendix

Appendix 1: The R and Python scripts used to preprocess the data.

Python script:

```
import pandas as pd
import numpy as np
pd.set_option('display.max_colwidth', None)

# lets print multiple outputs in cell, not only the last one
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"

# load data
df = pd.read_csv('./data/covid-data.csv')

# make mask data (from
↪ https://www.cureus.com/articles/93826-correlation-between-mask-compliance-and-covid-19-outcomes-
mask_data = {
    'location': ['Albania', 'Bosnia and Herzegovina',
↪ 'Bulgaria', 'Croatia', 'Czechia', 'Hungary', 'North
↪ Macedonia', 'Poland', 'Romania', 'Serbia', 'Slovakia',
↪ 'Slovenia', 'Belarus', 'Estonia', 'Latvia',
↪ 'Lithuania', 'Republic of Moldova', 'Ukraine',
↪ 'Austria', 'Belgium', 'Denmark', 'Finland', 'France',
↪ 'Germany', 'Greece', 'Ireland', 'Italy',
↪ 'Netherlands', 'Norway', 'Portugal', 'Spain',
↪ 'Sweden', 'Switzerland', 'United Kingdom', 'Northern
↪ Ireland'],
    'avg_mask_usage': [0.53, 0.40, 0.55, 0.29, 0.52, 0.77,
↪ 0.67, 0.72, 0.81, 0.54, 0.76, 0.69, 0.55, 0.64, 0.64,
↪ 0.74, 0.66, 0.67, 0.55, 0.71, 0.14, 0.46, 0.76, 0.57,
↪ 0.84, 0.71, 0.91, 0.51, 0.29, 0.84, 0.95, 0.05, 0.53,
↪ 0.62, 0.68],
}

df_mask = pd.DataFrame(mask_data)

# join mask data to covid data
df_covid = df.merge(df_mask, on='location')

# drop countries, that are not in mask data
df_covid = df_covid.dropna(subset=['avg_mask_usage'])

# sort dataframe by date
df_covid['date'] =
↪ pd.to_datetime(df_covid['date']).dt.tz_localize(None)
df_covid = df_covid.sort_values(by='date')
df_covid.to_csv('./data/covid_data_mask.csv')

prep = pd.read_csv('./preprocessed_covid_data.csv')
#prep
```

```

# listaks kaikki päivämäärät per location
setlist = list(prepare.groupby('location').agg({'date': lambda x:
    ↪ set(x)}).reset_index()['date'])

# kaikista listoista (tai oikeestaan muutetaan seteiksi)
    ↪ intersection
u = list(set.intersection(*setlist))

# karsitaan datasta parilliset päivät
u2 = []
for i in u:
    if i % 20 == 0:
        u2.append(i)

# --> locataan ne päivät dataframesta!
prep2 =
    ↪ prep.loc[prep['date'].isin(u2)].drop(columns=['Unnamed:
    ↪ 0'])
prep2.to_csv('./data/intersection_preprocessed_covid_data.csv')
prep2

# poistetaan randomilla vielä nää maat tuolta datasta
category =
    ↪ prep2.groupby('mask_category')['location'].apply(lambda x:
    ↪ list(x.unique()))).reset_index()

# kaikilla oli 43 791 rivii ja parillisilla 21 879 rivii

def flatten(l):
    return [item for sublist in l for item in sublist]

c1 = set(flatten(category.iloc[[0]]['location'].tolist()))
c2 = set(flatten(category.iloc[[1]]['location'].tolist()))
c3 = set(flatten(category.iloc[[2]]['location'].tolist()))

# poistetaan c1:stä intersection 1 ja 2
i1 = set.intersection(c1, c2)
# poistetaan c2:stä intersection 2 ja 3
i2 = set.intersection(c2, c3)

ca1 = list(c1 - i1)
ca2 = list(c2 - i2)
ca3 = list(c3)

# valitaan jokaisesta kategoriasta 6 ekaa maata
ca1 = ca1[:4]
ca2 = ca2[:4]
ca3 = ca3[:4]

allca = flatten([ca1, ca2, ca3])

# valitaan nämä maat ja korjataan niiden maski kategoriat

```

```

prep3 = pd.DataFrame(prepare2)
prep3['mask_category'] =
    ↪ prep3['mask_category'].mask(prepare3['location'].isin(ca1),
    ↪ 1)
prep3['mask_category'] =
    ↪ prep3['mask_category'].mask(prepare3['location'].isin(ca2),
    ↪ 2)
prep3['mask_category'] =
    ↪ prep3['mask_category'].mask(prepare3['location'].isin(ca3),
    ↪ 3)
prep3 = prep3.loc[prepare3['location'].isin(allca)]

prep3.to_csv('./data/intersection_preprocessed_covid_data.csv')

```

R script:

```

# Calculate the number of countries in each category
num_categories <- 3

# Add a new column "mask_category" to the dataframe
covid_data <- covid_data %>%
  mutate(mask_category = ntile(avg_mask_usage,
    ↪ num_categories))

# Print the number of rows in each category
table(covid_data$mask_category)

# Note! The code above created overlapping bins of categories,
# which were later fixed with Python

# Calculate the lowest and highest values of avg_mask_usage in
    ↪ each category
summary_by_category <- covid_data %>%
  group_by(mask_category) %>%
  summarize(min_avg_mask_usage = min(avg_mask_usage),
    max_avg_mask_usage = max(avg_mask_usage))

# Print the summary
print(summary_by_category)

# Count the number of countries in each category
count_by_category <- covid_data %>%
  group_by(mask_category) %>%
  summarise(num_countries = n_distinct(location))

# Print the summary
print(count_by_category)

column_names = c("location", "date", "mask_category",
    ↪ "total_deaths_per_million")

# Check for NA values in specified columns

```

```

rows_with_na <- covid_data %>%
  filter(rowSums(is.na(select(., one_of(column_names)))) > 0)

# Count the number of rows with NA values
num_rows_with_na <- nrow(rows_with_na)
cat("Number of rows with NA values:", num_rows_with_na, "\n")

# Remove rows with NA values from the dataframe
covid_data <- covid_data %>%
  filter(!rowSums(is.na(select(., one_of(column_names)))) > 0)

# List of columns to preserve
columns_to_preserve <- c("location", "date",
  ↪ "total_deaths_per_million", "mask_category")

# Select the columns
covid_data <- covid_data[, columns_to_preserve, drop = FALSE]

covid_data

# Calculate the numeric representation of dates starting from
  ↪ zero
covid_data <- covid_data %>%
  mutate(date = as.numeric(date - min(date)))

# Print the modified dataframe
print(covid_data)

# Transform data to GroupedData object
grouped_data <- groupedData(total_deaths_per_million ~ date |
  ↪ location, data = covid_data)

# Check the structure of grouped_data
head(grouped_data, 10)

grouped_data$mask_category <-
  ↪ as.factor(grouped_data$mask_category)
head(grouped_data, 10)

```

Appendix 2. The whole R code used in the report above concatenated into one script.

```

# Let us define a list of needed packages, compare it to the
  ↪ output from installed.packages()[,"Package"] and install
  ↪ the missing packages:
list.of.packages <- c("ggplot2", "dplyr", "brms", "cmdstanr",
  ↪ "nlme", "rstan", "bayesplot", "gridExtra", "tidybayes")
new.packages <- list.of.packages[!(list.of.packages %in%
  ↪ installed.packages()[,"Package"])]
if(length(new.packages)) install.packages(new.packages)

```

```

# Load the libraries
library(ggplot2)
library(dplyr)
library(brms)
library(cmdstanr)
library(nlme)
library(rstan)
library(bayesplot)
library(gridExtra)
library(tidybayes)

# Globally specify cmdstan backend for brms
options(brms.backend="cmdstanr")
# Tell brms to cache results if possible
options(brms.file_refit="on_change")

# Load the data
covid_data <- read.csv("data/covid_data_mask.csv")

# Extract unique average values for each country
unique_avg_values <- covid_data %>%
  select(location, avg_mask_usage) %>%
  distinct()

# Create a plot of average mask usage by country
ggplot(unique_avg_values, aes(x = location, y =
  ↪ avg_mask_usage, fill = location)) +
  geom_bar(stat = "identity") +
  labs(title = "Average Mask Usage by Country",
    x = "Country",
    y = "Average Mask Usage") +
  scale_fill_viridis_d(option = "plasma", direction = -1) + #
  ↪ Custom viridis palette
  theme_minimal() +
  theme(axis.text.x = element_blank())

# Convert the "date" column to a date format
covid_data$date <- as.Date(covid_data$date)

# Plot the total number of deaths per million people in
  ↪ different countries
ggplot(covid_data, aes(x = date, y = total_deaths_per_million,
  ↪ color = location)) +
  geom_line() +
  labs(title = "Covid-19 Deaths per Million by Country",
    x = "Date",
    y = "Deaths per Million") +
  theme_minimal()

preprocessed_data <-
  ↪ read.csv("data/intersection_preprocessed_covid_data.csv")

```



```

ggplot(preprocessed_data,aes(x=total_deaths_per_million)) +
  ↪ geom_histogram(fill ="lightblue",color ="black") +
  ↪ labs(title ="Mortality Histogram",x ="Mortality (per
  ↪ million)",y ="Frequency")

max_days = 900
data_to_use = preprocessed_data

clipped_covid_data <- data_to_use %>%
  group_by(location) %>%
  filter(date < max_days) %>%
  ungroup()

# Print or use the selected_data dataframe
print(clipped_covid_data)

# Count the number of countries in each category
count_by_category <- data_to_use %>%
  group_by(mask_category) %>%
  summarise(num_countries = n_distinct(location))

# Print the summary
print(count_by_category)

# Transform data to GroupedData object
clipped_covid_data <- groupedData(total_deaths_per_million ~
  ↪ date | location, data = clipped_covid_data)

# as.factor(...) function is used to convert the selected
  ↪ column to a factor variable
clipped_covid_data$mask_category <-
  ↪ as.factor(clipped_covid_data$mask_category)

# Check the structure of grouped_data
head(clipped_covid_data, 10)

ggplot(clipped_covid_data, aes(x = date, y =
  ↪ total_deaths_per_million, color =
  ↪ as.factor(mask_category), group = interaction(location,
  ↪ mask_category))) +
  geom_line() +
  labs(title = "Number of Deaths per Million by Country and
  ↪ Mask Category",
    x = "Days",
    y = "Deaths per Million (log)") +
  scale_color_manual(values = c("1" = "steelblue", "2" =
  ↪ "darkorange", "3" = 'darkred')) +
  scale_y_continuous(trans = 'log') + # Set y-axis to a
  ↪ logarithmic scale using natural logarithm
  theme_minimal()

```

```

custom_priors <- c(
  prior(normal(0, 5), coef = "date"),
  prior(normal(0, 200), coef = "mask_category2"),
  prior(normal(0, 200), coef = "mask_category3")
)

suppressMessages(
  custom_fit <- brms::brm(
    # This specifies the formula
    total_deaths_per_million ~ 1 + mask_category + date,
    # This specifies the dataset
    data = clipped_covid_data,
    # This specifies the observation model family
    family = "gaussian",
    # This passes the priors specified above to brms
    prior = custom_priors,
    # This causes brms to cache the results
    file = "additional_files/covid_data_custom_fit",

    # Use these for parallel processing
    cores = 8,
    threads = threading(8)
  )
)

pp_check(custom_fit)

pp_plot <- pp_check(custom_fit, type = "intervals_grouped",
  ↪ group = "mask_category") +
  theme_light() # Use theme_light to make the background
  ↪ lighter

# Save the plot to a file with specified dimensions
ggsave("pp_check_plot.png", plot = pp_plot, height = 10, width
  ↪ = 25)

summary(custom_fit)

png(file="convergence_1_1.png",
  width=1100, height=1100)
plot(custom_fit)[1]

suppressMessages(
  custom_fit2 <- brms::brm(
    # This specifies the formula
    total_deaths_per_million ~ 1 + mask_category + date +
    ↪ (date|location), # include a group effect in brms
    # This specifies the dataset
    data = clipped_covid_data,
    # This specifies the observation model family
    family = "gaussian", # Could be improved by trying
    ↪ "lognormal",

```

```

# This passes the priors specified above to brms
prior = custom_priors,
# This causes brms to cache the results
file = "additional_files/covid_data_custom_fit2",

# Use these for parallel processing
cores = 8,
threads = threading(8)
)
)

brms::pp_check(custom_fit2)

pp_plot <- pp_check(custom_fit2, type = "intervals_grouped",
  ↪ group = "mask_category") +
  theme_light() # Use theme_light to make the background
  ↪ lighter

# Save the plot to a file with specified dimensions
ggsave("customfit2_pp_check_plot.png", plot = pp_plot, height
  ↪ = 10, width = 25)

summary(custom_fit2)

png(file="convergence_2_1.png",
width=1100, height=1100)
plot(custom_fit2)[1]

loo1 <- loo(custom_fit)
loo2 <- loo(custom_fit2)

print(loo1)
print(loo2)

comparison <- loo_compare(loo1, loo2)

print(comparison)

# Define a range of priors for 'date' coefficient
prior_ranges_date <- list(
  prior(normal(0, 10), coef = "date"), # Wider prior
  prior(normal(0, 2), coef = "date") # Narrower prior
)

# Define a range of priors for 'mask_category2' coefficient
prior_ranges_mask2 <- list(
  prior(normal(0, 400), coef = "mask_category2"), # Wider
  ↪ prior
  prior(normal(0, 100), coef = "mask_category2") #
  ↪ Narrower prior
)

```

```
# Define a range of priors for 'mask_category3' coefficient
prior_ranges_mask3 <- list(
  prior(normal(0, 400), coef = "mask_category3"),      # Wider
  ↪ prior
  prior(normal(0, 100), coef = "mask_category3")        #
  ↪ Narrower prior
)
```

```
# Fit models with different priors
models <- list()
i <- 1
for (prior_date in prior_ranges_date) {
  for (prior_mask2 in prior_ranges_mask2) {
    for (prior_mask3 in prior_ranges_mask3) {
      priors <- c(prior_date, prior_mask2, prior_mask3)
      suppressMessages(
        model <- brm(
          total_deaths_per_million ~ 1 + mask_category + date,
          data = clipped_covid_data,
          family = "gaussian",
          prior = priors,

          # For parallel processing
          cores = 8,
          threads = threading(8)
        )
      )

      models[[i]] <- pp_check(model)
      i <- i + 1
    }
  }
}
```

```
gridExtra::grid.arrange(grobs = models)
```

```
# Fit models with different priors for our hierarchical model
models_2 <- list()
i <- 1
for (prior_date in prior_ranges_date) {
  for (prior_mask2 in prior_ranges_mask2) {
    for (prior_mask3 in prior_ranges_mask3) {
      priors <- c(prior_date, prior_mask2, prior_mask3)
      suppressMessages(
        model <- brm(
          total_deaths_per_million ~ 1 + mask_category + date +
          ↪ (date|location), # include a group effect in brms
          data = clipped_covid_data,
          family = "gaussian",
          prior = priors,

          # For parallel processing
```

```

      cores = 8,
      threads = threading(8)
    )
  )

  models_2[[i]] <- pp_check(model)
  i <- i + 1
}
}
}

```

```

gridExtra::grid.arrange(grobs = models_2)

```

```

(category_fit <- clipped_covid_data %>%
  group_by(mask_category) %>%
  add_predicted_draws(custom_fit2) %>%
  ggplot(aes(x = date, y = total_deaths_per_million, color =
    ↪ ordered(mask_category), fill = ordered(mask_category)))
    ↪ +
  stat_lineribbon(aes(y = .prediction), .width = c(.95, .80,
    ↪ .50), alpha = 1/4) +
  geom_point(data = clipped_covid_data) +
  scale_fill_brewer(palette = "Set2") +
  scale_color_brewer(palette = "Dark2") +
  theme_bw() +
  ylab("Total deaths per million\n") +
  xlab("\nDay") +
  theme_bw() +
  theme(legend.title = element_blank()))

```